COURSEWORK ASSIGNMENT SUBMISSION

Name: Daksh Srivastava **Roll No:** 150096725087
Subject: Software Engineering & Project Management **Day:** Day 2 — Software Development Life Cycle (SDLC)

Section 11: Classroom Activity — Map the SDLC

Scenario: Launching the College Event Management System (CEMS) portal for the annual technical festival (12 events, strict capacity limits, mobile web access required, 3-week deadline).

1. Problem Statement

Campus technical festival coordinators currently face severe operational bottlenecks and registration errors due to decentralized, manual registration methods. The objective is to design, develop, test, and deploy a responsive web portal within three weeks that centralizes event discovery, automates registration validation, enforces strict per-event seat quotas, and generates real-time administrative reports.

2. Stakeholder Analysis

Stakeholder	Role / Responsibility	Critical Need
Festival Convener	Overall event scheduling and management	Real-time dashboard showing aggregate registration counts across all 12 events.
Student Registrant	End user participating in events	Mobile-friendly registration interface with immediate digital confirmation.
Technical Coordinator	Enforcing event rules and team sizes	Automated enforcement of minimum and maximum team member constraints.
Campus IT Administrator	Infrastructure stability and security	LDAP/Single Sign-On integration and zero system downtime during peak rush.

3. Requirements Specification

- **Functional Requirement 1:** The portal shall mandate student authentication via college email credentials prior to registration.
- **Functional Requirement 2:** The portal shall display the exact real-time seat availability for each of the 12 technical events.
- **Functional Requirement 3:** The portal shall prevent a student from registering for overlapping events scheduled at the identical time slot.
- **Functional Requirement 4:** The portal shall allow coordinators to export a consolidated, CSV-formatted attendee sheet.

- **Non-Functional Requirement 1 (Responsiveness):** The web interface shall be fully responsive across mobile viewport widths ranging from 360px to 420px without horizontal scroll.
- **Non-Functional Requirement 2 (Availability):** The system shall maintain 99.5% uptime across the active 3-week festival registration cycle.

4. Key Design Decisions

- **Decision 1 (Architecture):** Adopt a decoupled client-server architecture with a React-based Progressive Web App (PWA) frontend communicating via REST APIs with a Node.js/Express backend.
- **Decision 2 (Database & Concurrency):** Utilize a relational database (PostgreSQL) with row-level locking (`SELECT ... FOR UPDATE`) during registration commits to prevent race conditions when the last seat is claimed.
- **Decision 3 (Hosting):** Host backend services in containerized Docker instances behind an Nginx reverse proxy to balance inbound peak requests.

5. Verification & Test Cases

Test Case ID	Test Description	Input Data	Expected Output
TC-SDLC-01	Standard Registration	Valid student ID, Event #4 (Capacity 50/100)	Registration accepted, confirmation token rendered, seat count becomes 51.
TC-SDLC-02	Boundary Limit Test	Valid student ID, Event #2 (Capacity 100/100)	Registration rejected with message "Event capacity reached", seat count remains 100.
TC-SDLC-03	Concurrency Race Condition	2 simultaneous requests for 1 remaining seat	1 request succeeds (HTTP 200); 2nd request gracefully fails (HTTP 409 Conflict).

6. Deployment Readiness Checks

- **Check 1 (Database Migration & Backup):** Execute schema migration scripts on the staging database, verify foreign key indexes, and validate automated hourly database snapshot scripts.
- **Check 2 (Load Simulation & Smoke Test):** Simulate 200 concurrent user registration scripts using Apache Benchmark / k6; verify that error rates remain strictly below 0.1%.

7. Scope Management (Deferred Features)

- **Deferred 1: Automated Digital Certificate Generation:** Post-event certification is not required for initial registration. Moving this to Phase 2 preserves the critical 3-week path.
- **Deferred 2: In-App Chat and Discussion Forum:** Unnecessary operational complexity for MVP. Routine event announcements can be adequately handled via broadcast emails.

Section 15: Homework — Apply the SDLC to an Everyday Application

Selected Application: Zomato (Consumer Food Delivery and Restaurant Aggregation Platform).

- **1. Problem & Target Users:** Connects hungry consumers with local restaurants and delivery partners, managing food search, online ordering, live dispatch routing, and digital payments.
- **2. Requirements:** FR: (a) Geolocation-based restaurant filtering; (b) Cart management with item customization; (c) Live GPS tracking of delivery drivers. NFR: (a) Payment tokenization meeting PCI-DSS standards; (b) Order transmission to restaurant kitchen display within 2.5 seconds.
- **3. Planning Decision:** Adopting an iterative, microservice-oriented development strategy to decouple order placement from delivery driver allocation.
- **4. Design Decision & Rationale:** Deploying WebSockets for bi-directional live driver tracking. Rationale: Periodic HTTP polling at scale (millions of users) overwhelms web servers; WebSockets maintain lightweight, persistent connection state.
- **5. Normal & Boundary Tests:** Normal: Successfully place an order with an active credit card and receive restaurant acknowledgement. Boundary: Attempt to checkout with a promo code valid only for orders above Rs. 499 when cart value is exactly Rs. 499.00 vs Rs. 498.99.
- **6. Safe Deployment Strategy:** Canary deployment: Route 5% of incoming user traffic in a single city to the newly deployed release, monitor crash rates and checkout conversion for 2 hours, then progressively roll out to 100%.

DAKSH